National MCP Servers for Physical AI Oncology Clinical Trial Systems

Kevin Kawchak
Chief Executive Officer
ChemicalQDevice
kevink@chemicalqdevice.com

March 9, 2026

Abstract

Prior oncology clinical trial infrastructure has been fragmented, site-specific, and inefficient across the critical dimensions of interoperability, auditability, privacy, and deployment. Each trial site has historically operated its own isolated systems for electronic health records, imaging archives, and audit logging, resulting in inconsistent data formats, duplicated regulatory effort, and limited cross-site collaboration. This paper presents the National MCP Physical AI Oncology Trials system, a proposed end-to-end architecture comprising five Model Context Protocol (MCP) servers that address these gaps through standardized, federated, and safety-governed capabilities. The five servers (Authorization, FHIR Clinical Data, DICOM Imaging, Audit Ledger, and Provenance) expose 23 tools across five hierarchical conformance levels, validated by 668 test functions. The system integrates AI and robotics advances intended to improve patient safety and clinical effectiveness, including emergency stop coordination, procedure state machines, and multi-party approval checkpoints. Quantitative analysis of the repository (version 1.1.0) demonstrates 381 files across 88 directories, 34 integration adapters, 8 safety modules, 13 JSON schemas, and dual-language SDKs in Python and TypeScript. This proposed end-to-end MCP Physical AI oncology trial system provides the foundation for national-scale standardization of robotic oncology clinical trials.

Keywords Model Context Protocol · Physical AI · Oncology Clinical Trials · Federated Learning · Robotic Surgery

Contents

1 Introduction **4**
1.1 Fragmentation in Prior Trial Systems . . . 4
1.2 The Model Context Protocol Opportunity . . . 4
1.3 Contributions of This Work . . . 4
1.4 Prior System Architecture . . . 5
1.5 Regulatory Context . . . 5

2 Methods **5**
2.1 Development Process and Timeline . . . 5
2.1.1 Repository Evolution . . . 6
2.1.2 Version History and AI Contributions . . . 6
2.1.3 Development Metrics . . . 6
2.2 Prompt Engineering Methodology . . . 6
2.3 Peer Review Process . . . 7

3 Results **7**
3.1 Five-Server MCP Architecture . . . 7
3.1.1 Authorization Server . . . 7
3.1.2 FHIR Clinical Data Server . . . 8
3.1.3 Audit Ledger Server . . . 8
3.1.4 Provenance Server . . . 8
3.1.5 DICOM Imaging Server . . . 8
3.1.6 Server Architecture Diagram . . . 8
3.1.7 Complete Tool Inventory . . . 9
3.2 Conformance Levels and Profiles . . . 9
3.3 Safety Modules for Physical AI . . . 10
3.4 Test Coverage and Validation . . . 10
3.5 Integration Adapters . . . 11
3.6 Repository Scale . . . 11
3.7 Deployment Infrastructure . . . 11
3.8 SDK and Developer Tooling . . . 11
3.9 Schema and Specification Infrastructure . . . 12
3.10 Emergency Stop Implementation . . . 12
3.11 Interoperability Testbed . . . 13

4 Discussion **13**
4.1 The Five-Server Architecture as a National Standard . . . 13
4.2 Cross-Trial and Cross-Robot Interoperability . . . 13
4.3 Patient Safety Through AI and Robotics Governance . . . 14

4.4 Federated Learning and Multi-Site Collaboration 14
4.5 Robot Procedure Data Flow 14
4.6 National Deployment Topology 15
4.7 Governance and Standards Framework 16
4.8 Comparison with Prior Infrastructure 16
4.9 CI/CD and Quality Assurance 16

5 Limitations and Future Work **17**
5.1 Human Limitations 17
5.2 Claude Code Limitations 17
5.3 OpenAI Limitations 17
5.4 Future Work 18

6 Conclusion **18**

1 Introduction

Oncology clinical trials represent one of the most complex operational domains in modern healthcare, requiring coordination among clinical sites, sponsors, contract research organizations (CROs), regulatory agencies, and increasingly, autonomous robotic systems. The prior trial infrastructure evolved organically over decades, producing a patchwork of site-specific implementations that created significant barriers to interoperability, auditability, and scalability.

1.1 Fragmentation in Prior Trial Systems

The previous oncology trial infrastructure suffered from several structural limitations that hindered both efficiency and patient safety:

1. **Site-specific data silos.** Each clinical site maintained its own electronic health record (EHR) system, picture archiving and communication system (PACS), and audit logging mechanism. Data exchange between sites required custom point-to-point integrations that were expensive to build and maintain.
2. **Inconsistent interoperability.** While standards such as HL7 FHIR R4 [1] and DICOM [2] existed, adoption was uneven. Many sites used proprietary data formats or older HL7 v2 interfaces, requiring complex translation layers.
3. **Manual audit processes.** Regulatory compliance with 21 CFR Part 11 [3] and HIPAA [4] was largely manual, relying on paper-based or spreadsheet-driven audit trails that were difficult to verify and vulnerable to tampering.
4. **Limited privacy controls.** De-identification of protected health information (PHI) was inconsistently applied, and cross-site data sharing lacked standardized privacy-preserving mechanisms such as federated learning [5] or differential privacy.
5. **No Physical AI integration standard.** As autonomous robotic systems entered oncology clinical settings for procedures such as biopsy, ablation, and radiation therapy, no standardized protocol existed for robot-to-clinical-system communication, safety governance, or audit logging of autonomous actions.

1.2 The Model Context Protocol Opportunity

The Model Context Protocol (MCP) [6] introduced in 2024 by Anthropic provides a standardized, open protocol for communication between AI agents and tool servers. MCP defines a structured interface where agents invoke named tools with typed parameters and receive typed responses, with built-in support for capability discovery, error handling, and extensibility.

This protocol architecture presents a natural fit for oncology clinical trials, where multiple actors (clinicians, robots, auditors, sponsors) need controlled access to clinical data through well-defined interfaces. The prior system lacked such a unifying protocol layer, forcing each integration to be built as a custom, bilateral connection.

1.3 Contributions of This Work

This paper presents the National MCP Physical AI Oncology Trials system (repository version 1.1.0), a proposed end-to-end architecture that addresses the limitations of prior trial infrastructure through five key contributions:

1. A five-server MCP architecture (`trialmcp-authz`, `trialmcp-fhir`, `trialmcp-dicom`, `trialmcp-ledger`, `trialmcp-provenance`) exposing 23 standardized tools.
2. Five hierarchical conformance levels enabling incremental adoption from basic authorization through full robotic procedure orchestration.
3. Eight safety modules specifically designed for Physical AI systems in clinical environments, including emergency stop coordination and procedure state machines.
4. A federated architecture supporting cross-site collaboration without centralized patient data, with privacy-preserving aggregation via FedAvg, FedProx, and SCAFFOLD [5, 7].
5. Comprehensive validation infrastructure comprising 668 test functions across unit, conformance, adversarial, security, and interoperability categories.

The system was developed iteratively across four repositories, progressing from single-site Physical AI frameworks [8] through federated learning [9] and MCP server prototyping [10] to the current national-scale standard [11].

1.4 Prior System Architecture

To understand why the national MCP system is necessary, it is helpful to examine the architecture that the prior trial infrastructure imposed. The following text diagram illustrates the point-to-point integration model that characterized the previous system:

```
PRIOR ONCOLOGY TRIAL ARCHITECTURE
(Point-to-Point Integrations)

Site A            Site B            Site C
+--------+        +--------+        +--------+
| EHR-A  |<------>| EHR-B  |        | EHR-C  |
+--------+   ^    +--------+   ^    +--------+
| PACS-A |   |    | PACS-B |   |    | PACS-C |
+--------+   |    +--------+   |    +--------+
| Audit-A|   |    | Audit-B|   |    | Audit-C|
+--------+   |    +--------+   |    +--------+
     ^       |         ^       |         ^
     |   Custom    Custom  Custom    Custom
     +-- APIs --+-- APIs --+-- APIs --+
                |          |
          +----------+  +----------+
          | Sponsor  |  |   CRO    |
          +----------+  +----------+

Each arrow = custom bilateral integration
N sites x M systems = O(N*M) integrations
```

In this model, every clinical site maintained its own EHR, PACS, and audit system with proprietary interfaces. Cross-site communication required custom API development for each pair of systems, creating a combinatorial explosion of integration effort. A trial spanning 50 sites with 3 systems each required up to $50 \times 3 = 150$ unique integration endpoints, each requiring individual maintenance, testing, and regulatory validation.

The national MCP system replaces this with a hub-and-spoke model where each site implements the same five MCP servers, and all actors (robots, clinicians, sponsors, CROs, auditors) interact through the standardized 23-tool interface. This reduces the integration complexity from $O(n \times m)$ to $O(n + m)$, where n is the number of sites and m is the number of actor types.

1.5 Regulatory Context

The prior trial infrastructure was designed before autonomous robotic systems became a practical reality in oncology. Existing regulatory frameworks, including ICH-GCP E6(R2) [12], 21 CFR Part 11 [3], and HIPAA [4], were developed for human-operated clinical systems. The introduction of Physical AI systems creates new requirements for:

- Audit logging of autonomous decisions and actions
- Safety governance for robotic procedures in clinical settings
- Privacy-preserving cross-site learning from robotic procedure outcomes
- Standardized capability declaration and validation for robotic systems

The national MCP system addresses these requirements through its five-server architecture, safety modules, and conformance framework, providing the infrastructure necessary for FDA-regulated oncology trials incorporating Physical AI systems.

2 Methods

2.1 Development Process and Timeline

The repository and this paper were constructed using an AI-assisted development methodology. Kevin Kawchak served as the prompt creator and system architect, directing AI code generation through structured prompts documented in `prompts.md`.

The primary AI system used for repository generation and this paper was **Anthropic Claude Code Opus 4.6**. Peer review was conducted using **OpenAI ChatGPT 5.4 Thinking**, with review documents archived under the `peer-review/` directory.

2.1.1 Repository Evolution

The system was developed across four repositories in sequence. Table 1 summarizes the development timeline:

Table 1: Repository development timeline and evolution.

Repository	Date Range	Days	Focus
`physical-ai-oncology-trials`	Feb 12–26	14	USL scoring, patient instructions, single-site Physical AI framework
`pai-oncology-trial-fl`	Feb–Mar 2026	10	Federated learning, privacy modules, regulatory compliance
`mcp-pai-oncology-trials`	Mar 2026	7	MCP server proof of concept, 5-server architecture, 23 tools
`national-mcp-pai-oncology-trials`	Mar 5–9, 2026	5	National standard, SDKs, conformance suite, governance

2.1.2 Version History and AI Contributions

The national repository progressed through the following versions, each driven by AI-assisted code generation:

- **v0.5.0 (March 5, 2026):** Initial repository scaffold with 5 MCP server implementations, 13 JSON schemas, 8 conformance profiles, 44 unit tests, and deployment configurations. Generated by Claude Code Opus 4.6 from a structured prompt.
- **v0.5.1 (March 6, 2026):** Added 34 integration adapters, 8 safety modules, interoperability testbed with 8 scenarios, benchmark suite, and expanded test coverage to 331 conformance tests plus 293 additional unit tests. This version addressed gaps identified by OpenAI ChatGPT 5.4 Thinking peer review.
- **v0.5.2 (March 6, 2026):** Incorporated peer review feedback: full server implementations with transport, routing, and middleware; persistent storage adapters (SQLite, PostgreSQL); Docker deployment stack; and Kubernetes manifests. AI fixes were applied across 2 days.
- **v1.0.0 (March 7, 2026):** Phase 5 release with Python and TypeScript SDKs, CLI tooling, code generation pipeline, stakeholder guides, operational documentation, and governance framework. Total: 381 files, 88 directories, approximately 69,800 lines.
- **v1.0.1 (March 8, 2026):** Documentation accuracy corrections: badge updates, test count reconciliation, CI pipeline documentation alignment (14 jobs).
- **v1.1.0 (March 9, 2026):** This paper, updated documentation, and release notes.

2.1.3 Development Metrics

Table 2 presents quantitative development metrics derived from release dates and version identifiers:

Table 2: Development metrics for the national repository (v0.5.0 through v1.1.0).

Metric	Value
Total span (v0.5.0 to v1.1.0)	5 days
Days with an AI update	5 days
Number of AI code generation sessions	6 versions
Days of AI code recommendations	4 days
Days of AI fixes (v0.5.2, v1.0.1)	2 days
Total span across all 4 repositories	36 days

2.2 Prompt Engineering Methodology

Each version was generated from a structured prompt that specified the target architecture, file structure, and quality requirements. The prompts are archived in `prompts.md` and follow a consistent pattern:

1. Define the target state (e.g., "implement 5 MCP servers with 23 tools").
2. Specify file paths and module structure.
3. List quality constraints (linting, testing, documentation).
4. Request CI/CD compatibility (ruff, pytest, GitHub Actions).

This paper was generated from a prompt specifying the 20-page structure, LaTeX template (arxiv-style [13]), section requirements, and formatting constraints. The prompt is archived in `prompts.md` under the v1.1.0 entry.

2.3 Peer Review Process

Peer review was conducted using OpenAI ChatGPT 5.4 Thinking, which analyzed the repository structure and provided recommendations documented in the `peer-review/` directory. Five review rounds by Claude (`claude-1-2026-03-06.md` through `claude-5-2026-03-06.md`) and two by OpenAI (`openai-1-2026-03-06.md`, `openai-2-2026-03-06.md`) addressed gaps in:

- Server implementation completeness (transport, routing, persistence)
- Deployment readiness (Docker, Kubernetes, Helm)
- Test coverage expansion (adversarial, security, interoperability)
- SDK and developer tooling

A response document (`response-A-2026-03-06.md`) tracked how each recommendation was addressed in subsequent versions.

3 Results

3.1 Five-Server MCP Architecture

The system implements five domain-specific MCP servers, each encapsulating a distinct responsibility within the oncology trial workflow. Together, they provide the complete protocol surface required for Physical AI integration with clinical trial systems.

Table 3: Five MCP servers, their tools, and conformance levels.

Server	Tools	Level	Primary Function
`trialmcp-authz`	5	1	Deny-by-default RBAC, SHA-256 token lifecycle, policy evaluation
`trialmcp-fhir`	4	2	FHIR R4 clinical data with HIPAA Safe Harbor de-identification
`trialmcp-dicom`	4	3	DICOM imaging queries, RECIST [14] measurements
`trialmcp-ledger`	5	1	Hash-chained 21 CFR Part 11 audit trail
`trialmcp-provenance`	5	4	DAG-based data lineage, SHA-256 fingerprinting
Total	**23**		

3.1.1 Authorization Server

The authorization server (`servers/trialmcp_authz/`) implements deny-by-default role-based access control (RBAC) as defined in ADR-007 (`docs/adr/007-deny-by-default-rbac.md`). It exposes five tools:

1. `authz_evaluate`: Evaluates whether a given role is permitted to invoke a specific tool.
2. `authz_issue_token`: Issues a scoped, time-limited token with SHA-256 hashing.
3. `authz_validate_token`: Validates a token's authenticity and expiration.
4. `authz_revoke_token`: Revokes a previously issued token.
5. `authz_list_policies`: Lists active policies for a given role or server.

The server is implemented in `servers/trialmcp_authz/server.py` (134 lines) with a policy engine (`policy_engine.py`) and token store (`token_store.py`).

3.1.2 FHIR Clinical Data Server

The FHIR server (`servers/trialmcp_fhir/`) provides clinical data access with mandatory HIPAA Safe Harbor de-identification. The de-identification pipeline (`deid_pipeline.py`) implements HMAC-SHA256 pseudonymization to maintain referential integrity across de-identified records. The four tools are:

- `fhir_read`: Read a single FHIR R4 resource by type and ID.
- `fhir_search`: Search FHIR resources with parameter filtering (limit: 100).
- `fhir_patient_lookup`: Look up a patient by pseudonymized identifier.
- `fhir_study_status`: Retrieve the status of a clinical study.

3.1.3 Audit Ledger Server

The ledger server (`servers/trialmcp_ledger/`) implements a hash-chained immutable audit trail satisfying 21 CFR Part 11 requirements [3]. The chain implementation in `chain.py` uses SHA-256 hashing with canonical JSON serialization. The following code excerpt shows the core hashing mechanism:

Listing 1: Hash-chained audit record computation from `servers/trialmcp_ledger/chain.py`.

```python
GENESIS_HASH = "0" * 64

def canonical_json(record: dict[str, Any]) -> str:
    filtered = {k: v for k, v in sorted(record.items())
                if k != "hash"}
    return json.dumps(filtered, sort_keys=True,
                      ensure_ascii=True)

def compute_audit_hash(record, prev_hash: str) -> str:
    canonical = canonical_json(record)
    payload = prev_hash + canonical
    return hashlib.sha256(
        payload.encode("utf-8")).hexdigest()
```

Each audit record contains: `audit_id`, `timestamp`, `server`, `tool`, `caller`, `parameters`, `result_summary`, `previous_hash`, and `hash`. The chain is verified by recomputing each hash and comparing it against the stored value.

3.1.4 Provenance Server

The provenance server (`servers/trialmcp_provenance/`) maintains a directed acyclic graph (DAG) of data lineage records, enabling both forward and backward lineage queries. The DAG implementation in `dag.py` supports five source types (`fhir`, `dicom`, `computed`, `external`, `federated`) and five action types (`read`, `transform`, `aggregate`, `transfer`, `derive`).

3.1.5 DICOM Imaging Server

The DICOM server (`servers/trialmcp_dicom/`) provides imaging data access with role-based permissions and support for RECIST 1.1 [14] measurements critical to oncology trial response evaluation. The server exposes four tools: `dicom_query` for searching imaging studies by patient or accession number, `dicom_retrieve_pointer` for obtaining secure references to image data without transferring pixel data, `dicom_study_metadata` for retrieving study-level DICOM metadata, and `dicom_recist_measurements` for querying tumor measurement data used in treatment response assessment.

The DICOM adapter (`dicom_adapter.py`) supports integration with industry-standard PACS systems through the 9 DICOM integration adapters in `integrations/dicom/`, including dcm4chee, Orthanc, and DICOMweb connectors.

3.1.6 Server Architecture Diagram

The following text diagram shows the complete five-server deployment topology with data flow paths:

```
NATIONAL MCP FIVE-SERVER DEPLOYMENT
+--------------------------------------------------+
|                SITE DEPLOYMENT                   |
|                                                  |
```

```
|  +-----------+  +-----------+  +-----------+      |
|  | trialmcp  |  | trialmcp  |  | trialmcp  |      |
|  | authz     |  | fhir      |  | dicom     |      |
|  | (5 tools) |  | (4 tools) |  | (4 tools) |      |
|  | RBAC,     |  | FHIR R4,  |  | DICOMweb, |      |
|  | Tokens    |  | HIPAA     |  | RECIST    |      |
|  +-----+-----+  +-----+-----+  +-----+-----+     |
|        |              |              |              |
|        +------+-------+------+-------+              |
|               |              |                      |
|        +------v------+ +----v--------+             |
|        | trialmcp    | | trialmcp    |             |
|        | ledger      | | provenance  |             |
|        | (5 tools)   | | (5 tools)   |             |
|        | SHA-256     | | DAG, W3C    |             |
|        | 21 CFR 11   | | PROV        |             |
|        +-------------+ +-------------+             |
|                                                    |
|  23 tools total across 5 servers                   |
+----------------------------------------------------+
```

3.1.7 Complete Tool Inventory

Table 4 provides the complete inventory of all 23 tools with their input parameters and response types:

Table 4: Complete 23-tool inventory across all five MCP servers.

Tool	Key Parameters	Response Type
`authz_evaluate`	role, tool	ALLOW/DENY decision
`authz_issue_token`	role, expires_in	Token with SHA-256 hash
`authz_validate_token`	token_hash	Valid/invalid status
`authz_revoke_token`	token_hash	Revocation confirmation
`authz_list_policies`	role, server	Policy list
`fhir_read`	resource_type, id	De-identified FHIR resource
`fhir_search`	resource_type, params	De-identified bundle
`fhir_patient_lookup`	pseudonym	De-identified patient
`fhir_study_status`	study_id	Study status object
`dicom_query`	patient_id, modality	Study result list
`dicom_retrieve_pointer`	study_uid	Secure image pointer
`dicom_study_metadata`	study_uid	DICOM metadata
`dicom_recist_measurements`	study_uid	RECIST measurements
`ledger_append`	server, tool, caller	Audit record with hash
`ledger_verify`	(none)	Chain validity status
`ledger_query`	server, tool, caller	Filtered audit records
`ledger_replay`	start_index, count	Record sequence
`ledger_export`	format	Full chain export
`provenance_record`	source_id, action, actor	Provenance record
`provenance_query_forward`	record_id, depth	Downstream lineage
`provenance_query_backward`	record_id, depth	Upstream lineage
`provenance_fingerprint`	data	SHA-256 fingerprint
`provenance_lineage`	source_id	Full lineage graph

3.2 Conformance Levels and Profiles

The standard defines five hierarchical conformance levels, each building upon the previous:

Table 5: Five hierarchical conformance levels with required servers and use cases.

Level	Name	Required Servers	Use Case
1	Core	authz + ledger	Basic authorization and audit
2	Clinical Read	Level 1 + fhir	FHIR clinical data access
3	Imaging	Level 2 + dicom	DICOM imaging queries
4	Federated Site	Level 3 + provenance	Multi-site data lineage
5	Robot Procedure	All 5 servers	Full autonomous clinical workflow

Eight conformance profiles are defined in `profiles/`, including five hierarchical profiles and three regulatory overlays (`country-us-fda.md`, `state-us-ca.md`, `state-us-ny.md`).

3.3 Safety Modules for Physical AI

The system provides eight safety modules in `safety/` specifically designed for robotic systems operating in clinical environments:

Table 6: Safety modules for Physical AI systems.

Module	Purpose
`estop.py`	Emergency stop signal propagation, abort workflow, post-abort evidence capture, recovery with re-authorization
`procedure_state.py`	Procedure state machine enforcement (IDLE, PREPARING, ACTIVE, PAUSED, COMPLETING, ABORTED)
`robot_registry.py`	Robot registration, capability validation, firmware version tracking
`task_validator.py`	Task order validation against safety constraints and robot capabilities
`gate_service.py`	Safety gate evaluation before procedure transitions
`approval_checkpoint.py`	Multi-party approval checkpoints (clinician + system + regulatory)
`site_verifier.py`	Site capability verification against conformance requirements

The emergency stop controller in `safety/estop.py` implements a four-state lifecycle (IDLE, TRIGGERED, ACKNOWLEDGED, RECOVERING) with signal propagation to all registered servers. This ensures that any authorized party can halt a robotic procedure immediately, with full state preservation and evidence capture.

3.4 Test Coverage and Validation

The repository contains 668 test functions across two test suites:

Table 7: Test coverage summary across unit and conformance suites.

Category	Tests	Validates
Unit (integrations)	205	All 34 integration adapters
Unit (safety)	88	All 8 safety modules
Unit (core server)	32	5 MCP server implementations
Unit (infrastructure)	12	Conformance harness, schema validator
Positive conformance	69	Correct behavior at Levels 1–3
Adversarial	57	AuthZ bypass, chain tampering, PHI leakage, replay attacks
Blackbox	63	Per-server external conformance
Security	39	SSRF prevention, token lifecycle, chain integrity
Interoperability	32	Cross-server trace, schema validation
Negative	26	Invalid input rejection, unauthorized access
Integration	13	Multi-server integration scenarios
Total	**668**	

3.5 Integration Adapters

The repository implements 34 integration adapter modules across six categories in `integrations/`:

Table 8: Integration adapter categories and module counts.

Category	Count	Key Adapters
FHIR	9	HAPI FHIR, SMART on FHIR, de-identification, terminology, patient filter
DICOM	9	dcm4chee, Orthanc, DICOMweb, RECIST, modality filter
Clinical	3	Scheduling, e-consent, provenance export
Federation	4	Coordinator, policy enforcement, secure aggregation
Identity	5	OIDC, mTLS, KMS, policy engine
Privacy	4	De-identification pipeline, privacy budget, data residency
Total	**34**	

3.6 Repository Scale

Table 9 summarizes the quantitative scope of the national repository at version 1.1.0:

Table 9: Repository quantitative summary (v1.1.0).

Category	Count	Category	Count
MCP servers	5	Safety modules	8
Tools (total)	23	Benchmark modules	5
JSON schemas	13	Certification tools	4
Conformance profiles	8	Deployment modes	6
Test functions	668	Interop scenarios	8
Integration adapters	34	Actor personas	6
Specification docs	9	Stakeholder guides	5
ADRs	7	Operational docs	5
Total files	381	Total directories	88
Lines of code (approx.)	69,800	Regulatory overlays	7

3.7 Deployment Infrastructure

The system supports six deployment configurations, enabling evaluation at any scale:

1. **Individual Docker containers:** Five Dockerfiles in `deploy/docker/` (e.g., `Dockerfile.authz`, `Dockerfile.fhir`).
2. **All-in-one Docker:** `Dockerfile.allinone` for single-container evaluation.
3. **Docker Compose (single-site):** `deploy/docker-compose.yml` for local deployment.
4. **Docker Compose (multi-site):** `deploy/docker-compose.multi-site.yml` for federated testing.
5. **Kubernetes manifests:** 7 manifests in `deploy/kubernetes/` for production clusters.
6. **Helm chart:** `deploy/helm/trialmcp/` with `Chart.yaml` and `values.yaml`.

Server configuration is managed through YAML files in `deploy/config/` (one per server), enabling site-specific customization of storage backends, authentication providers, and privacy settings.

3.8 SDK and Developer Tooling

The repository provides dual-language SDKs and developer tools to facilitate adoption:

- **Python SDK** (`sdk/python/trialmcp_client/`): 8 client modules with 4 middleware components (audit, auth, circuit breaker, retry). Installable via `pip install -e .` with `pyproject.toml`.

- **TypeScript SDK** (`sdk/typescript/src/`): 8 client modules with matching middleware stack. Configured via `package.json` and `tsconfig.json`.
- **CLI** (`tools/cli/trialmcp_cli.py`): Commands for `init`, `scaffold`, `validate`, `certify`, `schema diff`, and `config generate`.
- **Code generation** (`tools/codegen/`): Generators for Python models, TypeScript interfaces, and OpenAPI specifications from JSON schemas.

Both SDKs include role-scoped example scripts for all six actor types: robot agent, trial coordinator, data monitor, auditor, sponsor, and CRO.

3.9 Schema and Specification Infrastructure

The repository defines 13 machine-readable JSON schemas in `schemas/` using JSON Schema Draft 2020-12. These schemas serve as the normative contract between MCP server implementations and clients, enabling automated validation and code generation:

Table 10: JSON schemas organized by domain category.

Schema File	Category	Validates
`audit-record.schema.json`	Audit	Audit trail entries
`authz-decision.schema.json`	AuthZ	RBAC evaluation results
`capability-descriptor.schema.json`	Infra	Server capability declarations
`consent-status.schema.json`	Privacy	Patient consent state
`dicom-query.schema.json`	Imaging	DICOM query parameters
`error-response.schema.json`	Infra	Standardized error envelopes
`fhir-read.schema.json`	Clinical	FHIR resource read responses
`fhir-search.schema.json`	Clinical	FHIR search bundle responses
`health-status.schema.json`	Infra	Server health check responses
`provenance-record.schema.json`	Prov	DAG lineage records
`robot-capability-profile`	Robotics	Robot system capabilities
`site-capability-profile`	Infra	Site-level capabilities
`task-order.schema.json`	Robotics	Procedure task orders

Nine normative specification documents in `spec/` define the complete protocol surface, including `core.md` (protocol scope, design principles, 5 conformance levels), `actor-model.md` (6 actors with permission matrices), `tool-contracts.md` (23 tool signatures with input/output contracts), `security.md` (RBAC, mTLS, OIDC, SSRF prevention), and `privacy.md` (HIPAA Safe Harbor, de-identification, consent, data residency).

3.10 Emergency Stop Implementation

The emergency stop controller is a critical safety component. The following code excerpt from `safety/estop.py` shows the four-state lifecycle and signal propagation:

Listing 2: Emergency stop lifecycle from `safety/estop.py`.

```
class EStopStatus(Enum):
    IDLE = "IDLE"
    TRIGGERED = "TRIGGERED"
    ACKNOWLEDGED = "ACKNOWLEDGED"
    RECOVERING = "RECOVERING"

class EStopController:
    def trigger_estop(self, procedure_id, reason,
                      triggered_by, current_state=None):
        if self._status == EStopStatus.TRIGGERED:
            raise RuntimeError(
                "E-stop already active")
        signal = EStopSignal(
            signal_id=str(uuid.uuid4()),
            procedure_id=procedure_id,
            reason=reason,
            triggered_by=triggered_by,
            affected_servers=list(
                self._active_servers),
```

```
            preserved_state=current_state or {})
        self._active_signal = signal
        self._status = EStopStatus.TRIGGERED
        return signal
```

The e-stop lifecycle ensures: (1) only one e-stop can be active at a time, (2) all registered servers receive the stop signal, (3) procedure state is preserved for post-incident analysis, and (4) recovery requires explicit re-authorization. These properties are validated by 88 unit tests in `tests/test_safety.py`.

3.11 Interoperability Testbed

The interoperability testbed in `interop-testbed/` provides a multi-site simulation environment with 8 scenarios and 6 actor personas. Each scenario validates a specific cross-site interaction pattern:

1. `audit_replay.py`: Verifies audit trail replay across sites
2. `cross_site_provenance.py`: Validates cross-site provenance chain integrity
3. `partial_outage.py`: Tests graceful degradation during partial failures
4. `robot_workflow.py`: End-to-end robotic procedure workflow across servers
5. `schema_drift.py`: Tests schema version compatibility under evolution
6. `site_onboarding.py`: Validates new site onboarding workflow
7. `state_overlay.py`: Tests state-level regulatory overlay application
8. `token_exchange.py`: Validates cross-site token exchange protocols

The testbed uses Docker Compose (`interop-testbed/docker-compose.yml`) with mock services for EHR (`mock_ehr.py`), PACS (`mock_pacs.py`), and identity (`mock_identity.py`) to simulate realistic clinical environments.

4 Discussion

4.1 The Five-Server Architecture as a National Standard

The five-server MCP architecture represents a fundamental shift from the prior point-to-point integration model to a standardized, protocol-driven approach. Each server addresses a distinct domain concern, and together they provide a complete protocol surface for Physical AI oncology trials.

The architecture's significance lies in several properties:

Separation of concerns. By isolating authorization, clinical data, imaging, audit, and provenance into distinct servers, the architecture enables independent scaling, deployment, and upgrade of each capability. A site that only needs clinical data access (Level 2) can deploy the authorization, ledger, and FHIR servers without the imaging or provenance infrastructure.

Deny-by-default security. The authorization server implements explicit ALLOW rules evaluated against a permission matrix covering all 23 tools and 6 actor roles. Any request not matching an explicit rule is denied. This approach, formalized in ADR-007 (`docs/adr/007-deny-by-default-rbac.md`), provides defense in depth against unauthorized access to clinical data or robotic controls.

Immutable audit trail. The hash-chained ledger provides tamper-evident audit logging that satisfies 21 CFR Part 11 requirements for electronic records in clinical trials [3]. Every tool invocation across all five servers produces an audit record. The chain can be verified by any authorized party by recomputing hashes from the genesis block forward.

Privacy by design. The FHIR server enforces mandatory HIPAA Safe Harbor de-identification [4] before any clinical data leaves the server boundary. HMAC-SHA256 pseudonymization maintains referential integrity across records while preventing re-identification. The privacy integration adapters (`integrations/privacy/`) add privacy budget tracking and data residency enforcement.

4.2 Cross-Trial and Cross-Robot Interoperability

The standardized MCP interface enables interoperability along three dimensions:

1. **Between clinical sites.** Sites deploying the same five-server architecture can exchange audit chain summaries, provenance queries, and federated learning updates through standardized tool calls. The interoperability testbed (`interop-testbed/`) validates eight cross-site scenarios including `cross_site_provenance.py`, `token_exchange.py`, and `partial_outage.py`.
2. **Between robotic systems.** Any robot vendor implementing the MCP client interface can interact with any conforming trial site. The robot capability profile schema (`schemas/robot-capability-profile.schema.json`) and task order schema (`schemas/task-order.schema.json`) provide machine-readable contracts. The robot vendor guide (`docs/guides/robot-vendor.md`) documents the integration path.
3. **Between clinical trials.** Multiple trials at the same site share the same MCP server infrastructure, with RBAC policies scoping access per trial. This eliminates the prior model where each trial required its own isolated data access layer.

4.3 Patient Safety Through AI and Robotics Governance

The system's safety architecture is designed to protect patients at every stage of the robotic procedure lifecycle. The eight safety modules provide layered protection:

The **emergency stop controller** (`safety/estop.py`) enables any authorized party to halt a robotic procedure immediately. The e-stop signal propagates to all registered MCP servers, preserving the procedure state for post-incident analysis. Recovery requires explicit re-authorization by an authorized clinician.

The **procedure state machine** (`safety/procedure_state.py`) enforces valid transitions through six states: IDLE, PREPARING, ACTIVE, PAUSED, COMPLETING, and ABORTED. Invalid transitions (e.g., IDLE to ACTIVE without passing through PREPARING) are rejected, preventing unsafe procedure initiation.

The **multi-party approval checkpoints** (`safety/approval_checkpoint.py`) require concurrent approval from multiple parties (clinician, system, regulatory) before critical procedure transitions. This ensures that no single actor can authorize a high-risk robotic action unilaterally.

4.4 Federated Learning and Multi-Site Collaboration

The provenance server and federation integration adapters enable cross-site model training without centralizing patient data. The system supports three federated aggregation strategies:

- **FedAvg** [5]: Weighted averaging of model updates proportional to local dataset size.
- **FedProx**: Proximal term regularization for heterogeneous data distributions.
- **SCAFFOLD**: Variance reduction through control variates for improved convergence.

The federation adapters in `integrations/federation/` (coordinator, policy enforcement, secure aggregation, site harmonization) manage the cross-site coordination workflow while the privacy adapters enforce differential privacy budgets per query.

4.5 Robot Procedure Data Flow

The following text diagram illustrates the complete data flow for a Physical AI robot agent executing a clinical procedure within the national standard. Each numbered step corresponds to a specific MCP tool invocation, producing an audit record in the hash-chained ledger:

```
ROBOT AGENT         MCP SERVERS         CLINICAL SYSTEMS
-----------         -----------         ----------------
+--------+
| START  |
+---+----+
    |  1. Request Token
    v
+--------+    +--------------+
| AuthZ  |--->| trialmcp-    |  Issue scoped token
| Phase  |<---| authz        |  (role + expiry)
+---+----+    +--------------+
```

```
    |  2. Query Patient Data
    v
+--------+    +--------------+    +-----------+
| FHIR   |--->| trialmcp-    |--->| EHR       |
| Phase  |<---| fhir         |<---| (FHIR R4) |
+---+----+    +--------------+    +-----------+
    |  3. Retrieve Imaging
    v
+--------+    +--------------+    +-----------+
| DICOM  |--->| trialmcp-    |--->| PACS      |
| Phase  |<---| dicom        |<---| (DICOM)   |
+---+----+    +--------------+    +-----------+
    |  4. Execute Procedure
    v
+--------+
| ROBOT  |  Physical AI performs clinical task
| ACTION |  under safety constraints
+---+----+
    |  5. Record Audit
    v
+--------+    +--------------+
| Audit  |--->| trialmcp-    |  Hash-chained record
| Phase  |<---| ledger       |  (21 CFR Part 11)
+---+----+    +--------------+
    |  6. Record Provenance
    v
+--------+    +--------------+
| Prov.  |--->| trialmcp-    |  DAG lineage entry
| Phase  |<---| provenance   |  (SHA-256)
+---+----+    +--------------+
    v
+--------+
|  DONE  |
+--------+
```

This six-step workflow ensures that every robotic action is authorized (step 1), informed by clinical data (steps 2–3), executed under safety governance (step 4), and fully auditable (steps 5–6).

4.6 National Deployment Topology

The national standard envisions a three-tier deployment topology that scales from individual sites to regional aggregations to a national governance layer:

```
NATIONAL GOVERNANCE LAYER
+-------------------------------------------------------+
| Standards Body   | Conformance Registry               |
| Extension Mgmt   | Version Compatibility Policy       |
| Schema Registry  | Validation Services                |
+--------+---------+---------+--------+--------+------+
         |                   |                 |
+--------v------+  +--------v------+  +-------v------+
| REGION 1      |  | REGION 2      |  | REGION N     |
| (East)        |  | (Central)     |  | (West)       |
| 200+ Sites    |  | 300+ Sites    |  | 250+ Sites   |
| 5 Svrs/Site   |  | 5 Svrs/Site   |  | 5 Svrs/Site  |
+-------+-------+  +-------+-------+  +-------+------+
        |                  |                 |
        +------------------+-----------------+
                           |
```

```
+---------v----------+
| FEDERATED LAYER    |
| Aggregation        |
| Differential Priv. |
| Audit Merge        |
+--------------------+
```

Each site operates its own five-server deployment. The federated layer aggregates model updates using privacy-preserving techniques with differential privacy budgets, while audit chains are verified cross-site without exposing patient data. This architecture enables national-scale oncology trials while preserving patient privacy at each individual institution.

4.7 Governance and Standards Framework

The repository includes a comprehensive governance framework with 11 documents across `governance/` and `docs/governance/`. Key governance artifacts include:

- **Charter** (`governance/CHARTER.md`): Defines the standard's scope, decision-making process, and stakeholder roles.
- **Extension namespace** (`governance/EXTENSIONS.md`): Defines the `x-{vendor}` namespace for vendor-specific extensions that do not affect conformance.
- **Version compatibility** (`governance/VERSION_COMPATIBILITY.md`): Specifies SemVer rules, deprecation lifecycle, and backward compatibility guarantees.
- **Decision log** (`docs/governance/decision-log.md`): Records all architectural decisions with rationale and alternatives considered.
- **Roadmap** (`docs/governance/roadmap.md`): Tracks planned features and enhancements for future versions.

Seven architectural decision records (ADRs) in `docs/adr/` document the rationale behind key design choices, from the MCP protocol boundary (ADR-001) through the deny-by-default RBAC policy (ADR-007).

4.8 Comparison with Prior Infrastructure

Table 11 contrasts the prior oncology trial infrastructure with the proposed national MCP system:

Table 11: Comparison of prior trial infrastructure with the proposed national MCP system.

Dimension	Prior System	National MCP System
Architecture	Point-to-point custom integrations	5 standardized MCP servers, 23 tools
Interoperability	Site-specific, vendor-locked	Vendor-neutral MCP protocol
Audit trail	Manual, spreadsheet-driven	Hash-chained, tamper-evident (21 CFR 11)
Privacy	Inconsistent de-identification	Mandatory HIPAA Safe Harbor + HMAC-SHA256
Robot integration	No standard protocol	MCP tool contracts + safety modules
Cross-site learning	Central data repositories	Federated learning (FedAvg, FedProx)
Conformance	Ad hoc validation	5 levels, 668 test functions
Deployment	Custom per site	6 deployment modes (Docker to Helm)

4.9 CI/CD and Quality Assurance

The repository maintains quality through a comprehensive CI pipeline defined in `.github/workflows/ci.yml` with 14 jobs:

1. `lint-and-format`: Ruff linting and formatting across Python 3.10, 3.11, and 3.12
2. `integration-tests`: Multi-server integration validation

3. `adversarial-tests`: Security attack resistance
4. `schema-validation`: JSON Schema Draft 2020-12 validation
5. `schema-compatibility`: Schema metadata and field consistency
6. `contract-consistency`: Generated model alignment with schemas
7. `benchmark-smoke`: Performance benchmark verification
8. `sdk-python`: Python SDK import verification
9. `sdk-typescript`: TypeScript SDK compilation
10. `cli-smoke`: CLI command validation
11. `codegen-consistency`: Code generation pipeline verification
12. `security-scan`: Dependency audit and secret detection
13. `typescript-build`: Reference TypeScript compilation
14. `docs-lint`: Documentation completeness and link integrity

5 Limitations and Future Work

5.1 Human Limitations

The prompt-driven development methodology, while efficient for generating large-scale repository structure, has inherent limitations:

- **Domain expertise concentration.** The system was designed by a single author, which limits the breadth of clinical, regulatory, and engineering perspectives incorporated into the design.
- **Clinical validation gap.** While the architecture is designed to support real clinical workflows, it has not been deployed in an actual clinical trial setting. Validation with clinical stakeholders is required.
- **Prompt iteration overhead.** Each version required careful prompt construction and review of generated output. The prompt engineering process, while productive, required significant human attention to detail.

5.2 Claude Code Limitations

The Anthropic Claude Code Opus 4.6 system, while highly capable at code generation, exhibited several limitations:

- **Badge and count accuracy.** Version v1.0.0 contained incorrect badge counts (e.g., Unit Tests showing 44 instead of 337) that required manual identification and correction in v1.0.1.
- **Version string propagation.** Some files retained earlier version strings (v0.1.0) when the repository had advanced to v1.0.1, requiring systematic correction.
- **Documentation consistency.** Generated documentation sometimes contained internal inconsistencies (e.g., CI pipeline job counts) that required cross-referencing and correction.
- **Context window management.** Large repositories approach context window limits, requiring strategic prompt decomposition across multiple sessions.

5.3 OpenAI Limitations

The OpenAI ChatGPT 5.4 Thinking system used for peer review identified important architectural gaps but also exhibited limitations:

- **Repository access scope.** Peer review was conducted based on repository browsing rather than local execution, limiting the depth of code-level analysis.
- **Implementation specificity.** Some recommendations were architectural in nature without specific implementation guidance, requiring translation into actionable prompts.

5.4 Future Work

Future work for the author and industry includes increasing MCP Physical AI trial automation along several dimensions:

1. **Clinical pilot deployment.** Deploy the five-server architecture at one or more NCI-designated cancer centers to validate real-world interoperability with existing EHR and PACS systems.
2. **Robot vendor certification.** Develop a formal certification program for robotic systems seeking conformance with the national standard, including test harness integration and evidence package generation.
3. **Regulatory engagement.** Submit the standard specification to FDA for review as a recognized consensus standard under the FDA Voluntary Consensus Standards Recognition Program.
4. **Multi-site federated trials.** Conduct a proof-of-concept multi-site federated learning trial using the system's federation adapters to train tumor response prediction models across institutions.
5. **Real-time safety monitoring.** Extend the safety modules with real-time telemetry ingestion from robotic systems, enabling predictive safety interventions based on sensor data patterns.
6. **International expansion.** Extend the regulatory overlay framework to support EU MDR, UK MHRA, and other international regulatory regimes.
7. **Autonomous workflow expansion.** Increase the scope of automated procedures beyond the current safety-governed framework, incorporating advances in surgical robotics and AI-guided therapy.

6 Conclusion

This paper presents the National MCP Physical AI Oncology Trials system, a proposed end-to-end architecture for standardizing robotic oncology clinical trials at the national scale. The system addresses the fundamental limitations of prior oncology trial infrastructure, which relied on fragmented, site-specific, and inefficient point-to-point integrations that could not scale to meet the demands of modern AI-driven clinical research.

The prior system's inability to scale stemmed from its architectural foundation: each new site, each new robotic system, and each new clinical trial required custom integrations that multiplied complexity geometrically. With n sites and m robotic systems, the prior model required $O(n \times m)$ custom integrations. The national MCP system reduces this to $O(n + m)$ by providing a standardized protocol layer that both sites and robots implement against.

The five MCP servers (Authorization, FHIR, DICOM, Ledger, Provenance) provide the necessary capabilities organized by domain concern:

- **Interoperability** is achieved through standardized MCP tool contracts, 13 JSON schemas, and dual-language SDKs that enable any conforming implementation to communicate with any other.
- **Auditability** is provided by the hash-chained ledger server satisfying 21 CFR Part 11, with every tool invocation producing an immutable, tamper-evident audit record.
- **Privacy** is enforced through mandatory HIPAA Safe Harbor de-identification, HMAC-SHA256 pseudonymization, and federated learning architectures that keep patient data at the originating site.
- **Deployment** flexibility is supported through six configurations from single-container evaluation to Helm-templated Kubernetes clusters.

The system integrates AI and robotics advances intended to improve patient safety and clinical effectiveness. Eight safety modules govern the robotic procedure lifecycle, from emergency stop coordination through multi-party approval checkpoints. The five hierarchical conformance levels enable incremental adoption, allowing sites to begin with basic authorization and audit (Level 1) and progress to full robotic procedure orchestration (Level 5) as their capabilities mature.

Quantitative validation through 668 test functions across eight categories (including 57 adversarial tests and 39 security tests) demonstrates the system's commitment to safety-critical quality assurance. The 34 integration adapters spanning FHIR, DICOM, clinical, federation, identity, and privacy domains provide concrete interoperability with existing healthcare IT infrastructure.

The development process itself demonstrates the potential of AI-assisted software engineering for healthcare infrastructure, with the entire national repository (381 files, approximately 69,800 lines) generated through structured prompts to Anthropic Claude Code Opus 4.6 and peer-reviewed by OpenAI ChatGPT 5.4 Thinking over a span of 5 days (v0.5.0 through v1.1.0) and 36 days across all four repositories.

Patient benefit remains the central motivation for this work. By standardizing the infrastructure that governs how autonomous systems interact with clinical data, how audit trails are maintained, and how safety constraints are enforced, the national MCP system provides the foundation for oncology clinical trials that are safer, more transparent, and more effective than those conducted under prior fragmented infrastructure.

The repositories that contributed to this work are publicly available:

- `national-mcp-pai-oncology-trials` (v1.1.0) [11]
- `mcp-pai-oncology-trials` [10]
- `physical-ai-oncology-trials` [8]
- `pai-oncology-trial-fl` [9]

References

[1] HL7 International. HL7 FHIR release 4, 2019.

[2] NEMA. DICOM standard, 2024.

[3] U.S. Food and Drug Administration. 21 CFR part 11: Electronic records; electronic signatures, 2003.

[4] U.S. Department of Health and Human Services. Health insurance portability and accountability act (HIPAA), 1996.

[5] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[6] Anthropic. Model context protocol, 2024. Open standard for AI agent tool integration.

[7] Peter Kairouz, H. Brendan McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021.

[8] Kevin Kawchak. Physical AI oncology trials: End-to-end framework for robotic systems in clinical trials, 2026. GitHub: `https://github.com/kevinkawchak/physical-ai-oncology-trials`.

[9] Kevin Kawchak. PAI oncology trial FL: Federated learning for physical AI oncology trials, 2026. GitHub: `https://github.com/kevinkawchak/pai-oncology-trial-fl`.

[10] Kevin Kawchak. TrialMCP: MCP servers for physical AI oncology clinical trial systems, 2026. GitHub: `https://github.com/kevinkawchak/mcp-pai-oncology-trials`.

[11] Kevin Kawchak. National MCP standard for physical AI oncology clinical trials, 2026. GitHub: `https://github.com/kevinkawchak/national-mcp-pai-oncology-trials`.

[12] International Council for Harmonisation. ICH harmonised guideline: Integrated addendum to ICH E6(R1): Guideline for good clinical practice E6(R2), 2016.

[13] George Kour. arxiv-style: LaTeX style and template for paper preprints, 2024.

[14] E. A. Eisenhauer, P. Therasse, J. Bogaerts, et al. New response evaluation criteria in solid tumours: Revised RECIST guideline (version 1.1). *European Journal of Cancer*, 45(2):228–247, 2009.

Acknowledgments

The author would like to acknowledge Anthropic for providing access to Claude Code for repository and paper generations; and OpenAI for providing access to ChatGPT for peer-review.

Ethical Disclosures

The author of the article declares no competing interests.

Rights and Permissions

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author(s) and source are properly credited, a link to the Creative Commons license is provided, and any modifications made are indicated. To view a copy of this license, visit `https://creativecommons.org/licenses/by/4.0/`.

Cite This Article

Kawchak K. National MCP Servers for Physical AI Oncology Clinical Trial Systems. Zenodo. 2026; 10.5281/zenodo.18916731.